

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```
data= pd.read_csv('dataset.csv')

df=pd.DataFrame(data)

df.head(10)
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type
0	30669	Male	3.0	0	0	No	children
1	30468	Male	58.0	1	0	Yes	Private
2	16523	Female	8.0	0	0	No	Private
3	56543	Female	70.0	0	0	Yes	Private
4	46136	Male	14.0	0	0	No	Never_worked
5	32257	Female	47.0	0	0	Yes	Private
6	52800	Female	52.0	0	0	Yes	Private
7	41413	Female	75.0	0	1	Yes	Self-employed
8	15266	Female	32.0	0	0	Yes	Private
9	28674	Female	74.0	1	0	Yes	Self-employed

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 43400 entries, 0 to 43399
Data columns (total 12 columns):
#   Column              Non-Null Count  Dtype
---  -
0   id                   43400 non-null  int64
1   gender               43400 non-null  object
2   age                  43400 non-null  float64
3   hypertension         43400 non-null  int64
4   heart_disease        43400 non-null  int64
5   ever_married         43400 non-null  object
6   work_type            43400 non-null  object
```

```

7  Residence_type      43400 non-null object
8  avg_glucose_level  43400 non-null float64
9  bmi                41938 non-null float64
10 smoking_status      30108 non-null object
11 stroke              43400 non-null int64
dtypes: float64(3), int64(4), object(5)
memory usage: 4.0+ MB

```

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.shape # عدد الصفوف والاعمدة
```

```
(43400, 12)
```

```
df.isna().sum() # فحص القيم المفقودة
```

	0
id	0
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	1462
smoking_status	13292
stroke	0

```
dtype: int64
```

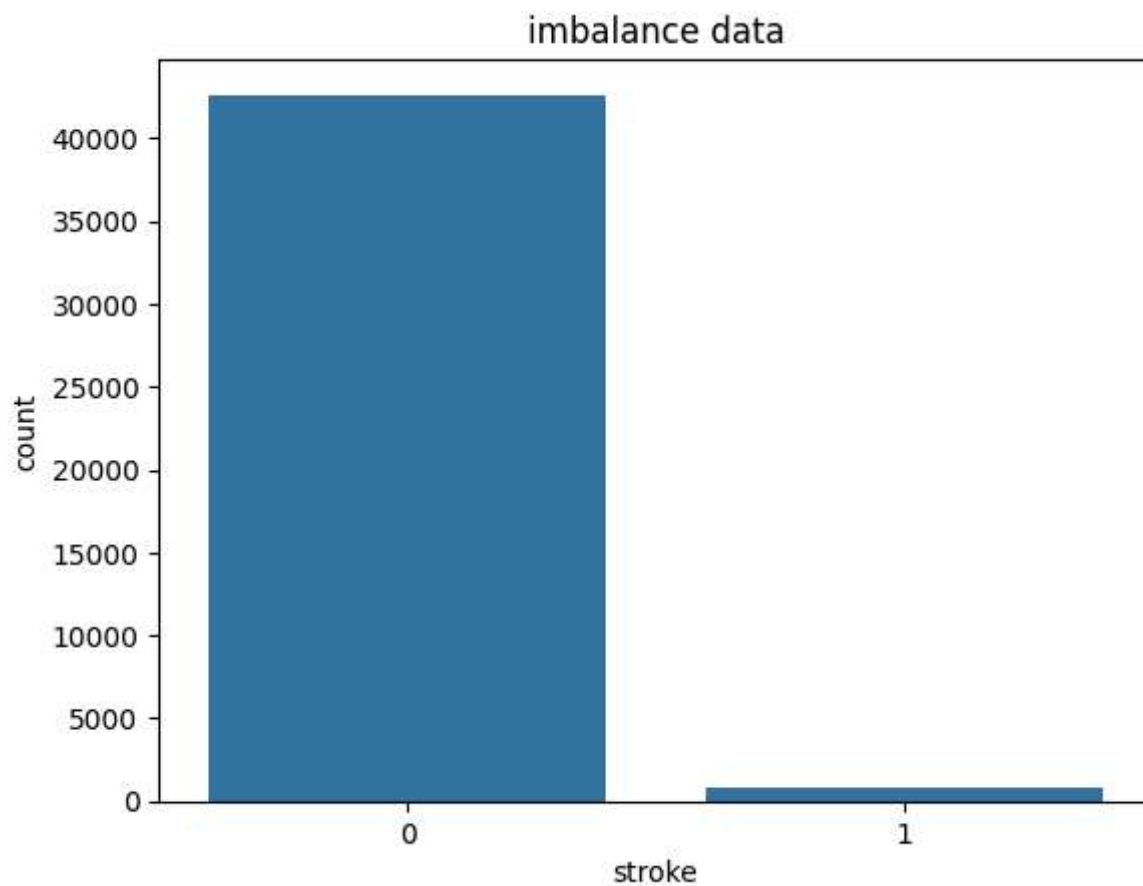
```
df['stroke'].value_counts()
```

	count
stroke	
0	42617
1	783

dtype: int64

تصوير توزيع البيانات

```
sns.countplot(x='stroke',data=df)
plt.title("imbalance data")
plt.show()
```



```
df = pd.DataFrame(data)

# 1. تنظيف القيم المفقودة
df['bmi'] = df['bmi'].fillna(df['bmi'].mean())
df['smoking_status'] = df['smoking_status'].fillna('Unknown')

# One-Hot Encoding
categorical_columns = ['gender', 'ever_married', 'work_type', 'Residence_type',
```

```

df = pd.get_dummies(df, columns=categorical_columns, drop_first=True, dtype=int)

# العمر
df['is_senior'] = (df['age'] > 60).astype(int)

# اكبر من 30 سنة
df['is_obese'] = (df['bmi'] > 30).astype(int)

# التي نريد التنبؤ بها stroke نحذف
X = df.drop('stroke', axis=1)
y = df['stroke']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# معالجة عدم التوازن
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

print(y_train.value_counts())

```

```

stroke
0    34098
1    34098
Name: count, dtype: int64

```

```

from sklearn.preprocessing import StandardScaler
Scaler=StandardScaler()
X_train=Scaler.fit_transform(X_train)
X_test=Scaler.transform(X_test)

```

logistic regression model

```

from sklearn.linear_model import LogisticRegression
logistic=LogisticRegression(max_iter=10000)

```

```
logistic.fit(X_train,y_train)
```

```

▼    LogisticRegression ⓘ ?
LogisticRegression(max_iter=10000)

```

```

# توقع ناتج بيانات الاختبار
y_pred = logistic.predict(X_test)

from sklearn.metrics import accuracy_score, confusion_matrix, classification_re

```

```
# حساب المقاييس
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_rep = classification_report(y_test, y_pred)

# طباعة النتائج
print(f"Accuracy: {accuracy}")
print("\nConfusion Matrix:")
print(conf_matrix)
print("\nClassification Report:")
print(class_rep)
```

Accuracy: 0.8783410138248848

Confusion Matrix:

```
[[7577  942]
 [ 114   47]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.89	0.93	8519
1	0.05	0.29	0.08	161
accuracy			0.88	8680
macro avg	0.52	0.59	0.51	8680
weighted avg	0.97	0.88	0.92	8680

```
from sklearn.tree import DecisionTreeClassifier
decision_tree = DecisionTreeClassifier()
```

```
decision_tree.fit(X_train, y_train)
```

▼ DecisionTreeClassifier ⓘ ?

```
DecisionTreeClassifier()
```

```
# توقع النتائج
y_pred_dt = decision_tree.predict(X_test)
```

```
# حساب المقاييس
accuracy_dt = accuracy_score(y_test, y_pred_dt)
conf_matrix_dt = confusion_matrix(y_test, y_pred_dt)
classification_rep_dt = classification_report(y_test, y_pred_dt)

# طباعة النتائج
print(f"Accuracy: {accuracy_dt}")
print("\nConfusion Matrix:")
print(conf_matrix_dt)
```

```
print("\nClassification Report:")
print(classification_rep_dt)
```

Accuracy: 0.9341013824884793

Confusion Matrix:

```
[[8090  429]
 [ 143   18]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.95	0.97	8519
1	0.04	0.11	0.06	161
accuracy			0.93	8680
macro avg	0.51	0.53	0.51	8680
weighted avg	0.97	0.93	0.95	8680

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
```

انشاء موديل

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

▼ RandomForestClassifier ⓘ ?

RandomForestClassifier(random_state=42)

استخدام الموديل المدرب لتوقع النتائج من بيانات الاختبار

```
y_pred = rf_model.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

accuracy_rf = accuracy_score(y_test, y_pred)
conf_matrix_rf = confusion_matrix(y_test, y_pred)
classification_rep_rf = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy_rf}")
print("\nConfusion Matrix:")
print(conf_matrix_rf)
print("\nClassification Report:")
print(classification_rep_rf)
```

Accuracy: 0.9574884792626728

Confusion Matrix:

```
[[8301  218]
 [ 151   10]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.97	0.98	8519
1	0.04	0.06	0.05	161
accuracy			0.96	8680
macro avg	0.51	0.52	0.51	8680
weighted avg	0.96	0.96	0.96	8680

```
from xgboost import XGBClassifier
```

تعريف وتدريب النموذج

```
xgb_model = XGBClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
xgb_model.fit(X_train, y_train)
```

▼

XGBClassifier

i ?

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=True, eval_metric=None, feature_types=None,
               feature_weights=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.1, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=100,
               n_jobs=None, num_parallel_tree=None, ...)
```

توقع النتائج

```
y_pred_xgb = xgb_model.predict(X_test)
```

حساب المقاييس

```
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
conf_matrix_xgb = confusion_matrix(y_test, y_pred_xgb)
classification_rep_xgb = classification_report(y_test, y_pred_xgb)
```

```
print(f"Accuracy: {accuracy_xgb}")
print("\nConfusion Matrix:")
print(conf_matrix_xgb)
print("\nClassification Report:")
print(classification_rep_xgb)
```

Accuracy: 0.9369815668202764

Confusion Matrix:

```
[[8114  405]
 [ 142   19]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.95	0.97	8519
1	0.04	0.12	0.06	161
accuracy			0.94	8680
macro avg	0.51	0.54	0.52	8680
weighted avg	0.97	0.94	0.95	8680

مقارنة دقة النماذج

```
import matplotlib.pyplot as plt

models = ['Logistic Regression', 'Decision Tree', 'Random Forest', 'XGBoost']

accuracies = [accuracy, accuracy_dt, accuracy_rf, accuracy_xgb]
plt.bar(models, accuracies, color=['blue', 'green', 'orange', 'purple'])

plt.ylabel('Accuracy')
plt.title('Model Accuracies')
plt.ylim(0.8, 1.0)
plt.show()
```


Model Accuracies

1.000

```
import pickle
```

```
# أولاً Scaler و rf_model تأكد من تشغيل خلايا تدريب  
pickle.dump(rf_model, open('stroke_model.sav', 'wb'))
```

```
pickle.dump(Scaler, open('scaler.sav', 'wb'))
```

